KONZEPTE DER IT · 475.007

KIT Theorie

Lernskript zu den Modulen Zahlendarstellung und Rechnen sowie Schaltwerke und Speicher

So liest du dieses Skript

- Der Haupttext stammt aus den Kursunterlagen (Skriptum, Übungsblatt 2, Weiterführende Übungen 4).
- Ergänzung Blau umrandete Kästen sind zusätzliche Erklärungen, Beispiele und Merkhilfen. Sie stammen nicht aus dem Skriptum.
- Typische Fehler Orange umrandet: Fehler, die in der Prüfung oft passieren.
- In Schaltbildern sind **rote Zahlen** Signalwerte (0 oder 1) in einem Rechenbeispiel.
- Die Beispielwerte zu Zeiten und Frequenzen in den Ergänzungen sind frei gewählt.

Zahlendarstellung und Rechnen

1. Bits, Wertigkeit, MSB und LSB
2. Die vier Darstellungen im Vergleich
3. Umrechnen: Horner-Schema und Wertigkeiten
4. Zweierkomplement bilden
5. Hexadezimaldarstellung
6. Halbaddierer, Volladdierer, Addierwerk
7. Subtraktion als Ersatzaddition
8. Carry, Overflow und Zero-Flag
9. Bitstellen hinzufügen (Vorzeichenerweiterung)
10. Festkommadarstellung
11. Binärpräfixe (kibi, mebi, gibi)

Schaltwerke und Speicher

1. Schaltnetz vs. Schaltwerk, RS-Flip-Flop
2. D-Latch und D-Flip-Flop
3. JK-Flip-Flop
4. Zeitangaben und Taktgeneratoren
5. Register
6. Halbleiterspeicher (RAM, ROM)
7. Zähler (synchron, asynchron, Ring)
8. Synchrone Schaltwerke entwerfen
9. ROM als Logik, maximale Taktfrequenz, CPLD/FPGA

Zahlendarstellung und Rechnen

Ein Rechner kennt nur Bits. In diesem Teil geht es darum, wie aus Bits Zahlen werden (Binär, Hex, Zweierkomplement), wie man sie von Hand umrechnet und wie ein Addierwerk damit rechnet. Am Ende steht die Frage, ob ein Ergebnis überhaupt noch in die vorhandenen Bits passt (Carry und Overflow).

Roter Faden: Immer zuerst klären, *wie viele Bit* es sind und *wie die Bits gelesen werden* (vorzeichenlos oder Zweierkomplement). Dasselbe Bitmuster bedeutet je nach Lesart etwas anderes.

Zahlendarstellung und Rechnen Relevanz mittel Quelle: Skript S. 40–41

Bits, Wertigkeit, MSB und LSB

Lernziele

- Ich kann die Wertigkeit jeder Bitstelle angeben (2^0 bis 2^7).
- Ich kann MSB und LSB benennen und weiß, was das LSB über gerade/ungerade sagt.
- Ich weiß, dass n Bit genau 2^n Bitmuster ergeben.

Im Dezimalsystem hat jede Stelle eine Wertigkeit: $793 = 7 \cdot 10^2 + 9 \cdot 10^1 + 3 \cdot 10^0$. Im Binärsystem ist es genau gleich, nur mit der Basis 2 statt 10. Es gibt nur die Ziffern 0 und 1. Eine Ziffer heißt **Bit** (binary digit).

Wertigkeit	2^7=128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1
Name	MSB							LSB

- **MSB** (Most Significant Bit) = ganz links, höchste Wertigkeit.
- **LSB** (Least Significant Bit) = ganz rechts. Vorzeichenlos gilt: LSB = 0 heißt gerade, LSB = 1 heißt ungerade.
- Schreibweise: Die Basis steht tiefgestellt. 11_{10} ist elf, 11_2 ist 2 + 1 = 3.

Warum ist die Bitanzahl so wichtig?
In der Mathematik sind Zahlen unendlich. Ein Rechner hat aber eine feste Anzahl Leitungen. Mit n Bit gibt es genau 2^n verschiedene Bitmuster, mit 4 Bit also 16. Was nicht in diese Muster passt, kann der Rechner nicht darstellen. Darum kommen bei jeder Aufgabe die Fragen „Mit wie vielen Bit?“ und „Passt das Ergebnis noch?“.

Ergänzung · Wie groß ist der Zahlenbereich?

Bit n	Bitmuster 2^n	größte vorzeichenlose Zahl $2^n - 1$
4	16	15
8	256	255
10	1 024	1 023
16	65 536	65 535
32	4 294 967 296	4 294 967 295

Merken: Die Zweierpotenzen bis 2^{10} auswendig können: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024. Fast jede Rechnung in der Prüfung benutzt sie. Die größte Zahl mit n Bit besteht aus lauter Einsen (8 Bit: 1111 1111 = 255).

Typische Fehler

- Die Stelle ganz rechts hat Wertigkeit $2^0 = 1$, nicht 2^1.
- Mit n Bit ist die größte vorzeichenlose Zahl $2^n - 1$ (nicht 2^n), weil die 0 auch ein Muster belegt.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Warum Basis 2? Ein Transistor kennt zuverlässig nur zwei Zustände (Spannung hoch/niedrig). Mehr Stufen wären störanfälliger. Nachfrage eines Prüfers: „Wie viele Bit brauche ich für k verschiedene Werte?“ → die kleinste Zahl n mit $2^n \geq k$, also $n = \lceil \log_2 k \rceil$. Für 1000 Werte: $2^9 = 512$ reicht nicht, $2^{10} = 1024$ reicht → 10 Bit.

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Skript S. 41–46

Die vier Darstellungen im Vergleich

Baut auf: Bits, Wertigkeit, MSB und LSB

Lernziele

- Ich kenne die Wertebereiche aller vier Darstellungen für n Bit.
- Ich kann ein Bitmuster in jeder Darstellung als Dezimalzahl lesen.
- Ich kann begründen, warum das Zweierkomplement Standard ist.

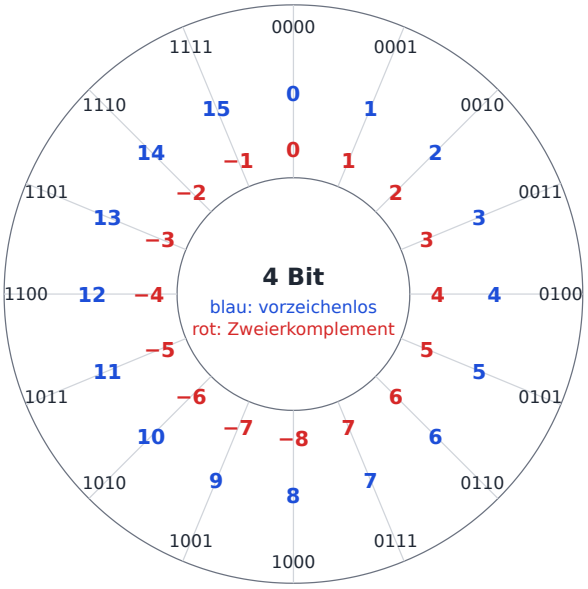
Dasselbe Bitmuster kann je nach Darstellung etwas ganz anderes bedeuten. Die Tabelle zeigt alle 16 Muster mit 4 Bit:

Bitmuster	vorzeichenlos	Vorzeichen-Betrag	Exzess-7	Zweierkomplement
0000	0	+0	−7	0
0001	1	1	−6	1
0010	2	2	−5	2
0011	3	3	−4	3
0100	4	4	−3	4
0101	5	5	−2	5
0110	6	6	−1	6
0111	7	7	0	7
1000	8	−0	1	−8
1001	9	−1	2	−7
1010	10	−2	3	−6
1011	11	−3	4	−5
1100	12	−4	5	−4
1101	13	−5	6	−3
1110	14	−6	7	−2
1111	15	−7	8	−1

Darstellung	Idee	Bereich mit n Bit	8 Bit
vorzeichenlos (C: unsigned int)	alle Bits haben ihre normale Wertigkeit	$0 \dots 2^n - 1$	0 … 255
Vorzeichen-Betrag	MSB ist das Vorzeichen (0 = +, 1 = −), die restlichen n−1 Bit sind der Betrag	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	−127 … +127
Exzess-E (offset binary)	zuerst vorzeichenlos lesen, dann den Exzess E abziehen	$-E \dots 2^n-1-E$	Exzess-127: −127 … +128
Zweierkomplement (C: int)	MSB hat die Wertigkeit -2^{n-1}, alle anderen sind normal	$-2^{n-1} \dots +(2^{n-1}-1)$	−128 … +127

Fallen: Bei Vorzeichen-Betrag gibt es die Null zweimal (+0 = 0000 und −0 = 1000). Im Zweierkomplement gibt es sie nur einmal. Deshalb ist der Bereich dort unsymmetrisch: Es gibt eine negative Zahl mehr (−8 … +7). Exzess-127 mit 8 Bit verwendet man bei IEEE 754 für den Exponenten. Als sinnvollen Exzess nimmt man bei n Bit 2^{n-1} oder $2^{n-1} - 1$.

Warum ist das Zweierkomplement Standard?
Positive Zahlen sehen gleich aus wie vorzeichenlose. Negative Zahlen haben immer MSB = 1, man erkennt das Vorzeichen also sofort. Der eigentliche Grund ist aber dieser: Man kann mit **demselben Addierwerk** rechnen wie bei vorzeichenlosen Zahlen und braucht keine Zusatzlogik. Bei Vorzeichen-Betrag oder Exzess bräuchte man Extra-Schaltungen.



Zahlenkreis für 4 Bit: außen das Bitmuster, blau vorzeichenlos, rot im Zweierkomplement.

Ergänzung · Der Zahlenkreis

Zählt man mit n Bit immer weiter, landet man nach dem größten Muster wieder bei 0000. Das Zählen läuft also im **Kreis**. Addieren heißt, im Kreis ein Stück zu laufen: bei vorzeichenloser Lesart immer vorwärts, im Zweierkomplement für einen positiven Summanden vorwärts und für einen negativen rückwärts.

- Läuft man vorzeichenlos über die Linie zwischen **15 und 0** (oben), liegt ein Übertrag vor: **Carry**.
- Läuft man im Zweierkomplement über die Linie zwischen **+7 und −8** (unten), ist das Ergebnis falsch: **Overflow**.

Beispiel 6 + 3: Von 6 geht man 3 Schritte vorwärts und landet bei 1001. Vorzeichenlos ist das 9 und stimmt. Im Zweierkomplement steht dort −7, man hat die untere Linie überquert: Overflow.

Beispiel 1101 + 1110 (vorzeichenlos 13 + 14, im Zweierkomplement −3 + (−2)): Das Ergebnis ist 1011 mit Übertrag. Vorzeichenlos geht man von 13 aus 14 Schritte vorwärts, überquert die obere Linie und landet bei 11. Das ist falsch (27 passt nicht in 4 Bit). Im Zweierkomplement geht man von −3 aus 2 Schritte rückwärts, überquert keine Linie und landet bei −5. Das stimmt.

Dasselbe Muster 1011 gelesen als …	Wert
vorzeichenlos	11
Vorzeichen-Betrag	−3
Exzess-7	+4
Zweierkomplement	−5

Typische Fehler

- Exzess vergessen abzuziehen.
- Vorzeichen-Betrag und Zweierkomplement verwechseln: 1011 ist −3 (VZ-Betrag) bzw. −5 (Zweierkomplement).
- Bereich 8-Bit-Zweierkomplement als −127…127 angeben (richtig: −128…127).

Zum Vertiefen: Hintergründe und Prüfernachfragen

Exzess-Darstellung hat einen praktischen Vorteil: Die Bitmuster sind in derselben Reihenfolge sortiert wie die Zahlen (0000 = kleinste, 1111 = größte). Darum verwendet IEEE 754 sie für den Exponenten – man kann Gleitkommazahlen dann fast wie Ganzzahlen vergleichen. Beim Zweierkomplement stimmt die Sortierung nicht (1111 = −1 ist „größer“ als 0111 = 7 als Bitmuster). Mögliche Prüfernachfrage: „Warum ist der Bereich im Zweierkomplement unsymmetrisch?“ → weil die 0 bei den „positiven“ Mustern (MSB = 0) mitzählt.

Zahlendarstellung und Rechnen | Relevanz hoch | Quelle: Skript S. 41–44 · Blatt 2 IV

Umrechnen: Horner-Schema und Wertigkeiten

Baut auf: Bits, Wertigkeit, MSB und LSB, Die vier Darstellungen im Vergleich

Lernziele

- Ich kann eine Dezimalzahl mit dem Horner-Schema in ein 8-Bit-Muster umrechnen.
- Ich kann ein Bitmuster über die Wertigkeiten in eine Dezimalzahl zurückrechnen.
- Ich kann negative Zahlen in Vorzeichen-Betrag und Exzess umrechnen.
- Ich prüfe vorher, ob die Zahl überhaupt darstellbar ist.

Binär → dezimal: Wertigkeiten addieren

Werkzeug: Unter jedes Bit die Wertigkeit schreiben und nur dort addieren, wo eine 1 steht. Man multipliziert ja nur mit 1 oder mit 0.

Wertigkeit	128	64	32	16	8	4	2	1
Bit	1	0	1	1	0	0	1	1

$1011\ 0011_2 = 128 + 32 + 16 + 2 + 1 = \boxed{179_{10}}$

Im Zweierkomplement ist es dasselbe, nur hat das MSB die Wertigkeit −128:

Wertigkeit	−128	64	32	16	8	4	2	1
Bit	1	0	1	1	0	1	0	1

$1011\ 0101_2 = -128 + 32 + 16 + 4 + 1 = \boxed{-75_{10}}$

Vorzeichen-Betrag, 4 Bit: 1011_2 → das MSB ist 1, also negativ. Der Betrag ist 011 = 2 + 1 = 3 → $\boxed{-3_{10}}$

Exzess-7, 4 Bit: $1011_2 = 8 + 2 + 1 = 11$ (vorzeichenlos), dann 11 − 7 = $\boxed{4_{10}}$

Dezimal → binär: Horner-Schema

> **Was wollen wir?** Herausfinden, welche Zweierpotenzen in der Zahl stecken.

> **Werkzeug:** Immer wieder durch 2 teilen. Der Rest (0 oder 1) ist jeweils das nächste Bit, angefangen beim LSB.
>
> **Wie genau?** Die Reste **von unten nach oben** lesen und dann vorne mit Nullen auf die gewünschte Bitanzahl auffüllen.

38 : 2 = 19	Rest 0 ← LSB
19 : 2 = 9	Rest 1
9 : 2 = 4	Rest 1
4 : 2 = 2	Rest 0
2 : 2 = 1	Rest 0
1 : 2 = 0	Rest 1 ← MSB, Ende weil 0

Von unten nach oben gelesen ergibt das 100110_2, mit 8 Bit `0010 0110`$_2$. Probe: 32 + 4 + 2 = 38 ✓

> **Warum von unten nach oben?**
> Der erste Rest sagt, ob die Zahl gerade ist. Das ist das LSB, also das Bit ganz rechts. Jede weitere Division schiebt die Zahl um eine Stelle nach rechts. Der letzte Rest ist deshalb das höchste Bit und kommt ganz nach links.

Negative Zahlen umrechnen

- **Vorzeichen-Betrag:** −23 → den Betrag 23 mit Horner umrechnen: 10111, mit 7 Bit 001 0111. Davor das MSB 1 setzen → `1001 0111`.
- **Exzess-127:** −112 → zuerst den Exzess addieren: −112 + 127 = 15. Dann 15 vorzeichenlos umrechnen → `0000 1111`.
- **Zweierkomplement:** den Betrag umrechnen und dann das Zweierkomplement bilden (siehe A4).
- Vorher immer prüfen, ob die Zahl in den Bereich passt. Ein Beispiel: 214 geht im 8-Bit-Zweierkomplement nicht, weil das Maximum 127 ist.

Ergänzung · Zweiter Weg: Zweierpotenzen abziehen

Wenn du die Zweierpotenzen auswendig kennst, geht es auch ohne Division. Von der größten passenden Potenz an abwärts fragt man bei jeder Potenz: Passt sie noch hinein? Wenn ja, schreibt man eine 1 und zieht sie ab, sonst eine 0.

```
179:  128 passt  → 1  (Rest 51)
       64 passt nicht → 0
       32 passt  → 1  (Rest 19)
       16 passt  → 1  (Rest 3)
        8 passt nicht → 0
        4 passt nicht → 0
        2 passt  → 1  (Rest 1)
        1 passt  → 1  (Rest 0)   →  1011 0011
```

Welche Methode wann? Zum Ablesen: Wertigkeiten addieren. Bei kleinen Zahlen: Zweierpotenzen abziehen. Bei großen oder unübersichtlichen Zahlen: Horner (durch 2 teilen). Beide Wege liefern dasselbe, die Probe über die Wertigkeiten ist immer möglich.

Typische Fehler

- Reste in falscher Reihenfolge gelesen.
- Führende Nullen vergessen (die Angabe verlangt 8 Bit!).
- Zahl ist im Bereich gar nicht darstellbar (z. B. 214 im 8-Bit-Zweierkomplement).

Zum Vertiefen: Hintergründe und Prüfernachfragen

„Horner-Schema“ heißt das Verfahren, weil eine Binärzahl als verschachtelte Klammer geschrieben werden kann: 38 = ((((1·2 + 0)·2 + 0)·2 + 1)·2 + 1)·2 + 0. Teilt man durch 2, schält man von innen nach außen die letzte Ziffer ab – deshalb ist der erste Rest das LSB. Gegenrichtung ohne Tabelle: Man liest von links und rechnet „mal 2, plus nächstes Bit“: 1 → 2 → 4 → 9 → 19 → 38.

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Skript S. 44–46

Zweierkomplement bilden

Baut auf: Die vier Darstellungen im Vergleich, Umrechnen: Horner-Schema und Wertigkeiten

Lernziele

- Ich kann das Zweierkomplement bilden (invertieren und +1).
- Ich kann eine negative Dezimalzahl in 8-Bit-Zweierkomplement umrechnen.
- Ich kann ein Zweierkomplement-Muster mit MSB = −128 zurückrechnen.
- Ich kann begründen, warum „invertieren + 1“ das Vorzeichen wechselt.

```
alle Bits invertieren, dann + 1 (ein Übertrag aus dem MSB fällt weg)
```

Beispiel: Gesucht ist −38 im 8-Bit-Zweierkomplement. −38 liegt in −128 … 127, passt also ✓

1. Betrag mit Horner umrechnen

38_{10} = 0010 0110

2. Jedes Bit umdrehen

Aus 0 wird 1 und aus 1 wird 0.

3. 1 addieren

```
  0010 0110   (+38)
  1101 1001   (alle Bits invertiert)
+        1    (plus eins)
  ---------
  1101 1010   (−38)
```

Probe: −128 + 64 + 16 + 8 + 2 = −38 ✓. Umgekehrt funktioniert es genauso: Bildet man von 1101 1010 das Zweierkomplement, kommt 0010 0110 = +38 heraus.

Sonderfall Null: Aus 0000 0000 wird invertiert 1111 1111, plus 1 ergibt das 1 0000 0000. Der Übertrag fällt weg, und es bleibt 0 ✓

Warum funktioniert „invertieren + 1“?
Eine Zahl plus ihre invertierte Version ergibt immer 1111 1111, also −1 im Zweierkomplement. Daraus folgt: x + invertiert(x) = −1. Zählt man auf beiden Seiten 1 dazu, steht da x + (invertiert(x) + 1) = 0. Der Teil in der Klammer ist also genau −x.

Begriffe: „Zweierkomplement“ ist die *Darstellung*. „Das Zweierkomplement bilden“ ist der *Algorithmus* für den Vorzeichenwechsel.

Ergänzung · Schnelltrick zur Kontrolle

Das Zweierkomplement lässt sich auch von rechts ablesen: Von rechts alle Bits bis **einschließlich der ersten 1** unverändert abschreiben, alle Bits links davon invertieren.

```
+38 = 0010 0110   → rechts: "10" bleibt, links "0010 01" invertieren
−38 = 1101 1010

+104 = 0110 1000  → rechts: "1000" bleibt, links "0110" invertieren
−104 = 1001 1000    Probe: −128 + 16 + 8 = −104 ✓
```

Das ist nur eine Abkürzung zum Gegenrechnen. In der Prüfung zeigst du den Weg „invertieren + 1“, weil er dem Addierwerk entspricht.

Typische Fehler

- Nur invertiert, +1 vergessen (das wäre das Einerkomplement).
- +1 an der falschen Stelle addiert (immer beim LSB).
- Positive Zahlen auch „komplementiert“ – positive bleiben wie vorzeichenlos.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Mathematisch rechnet ein n-Bit-Addierwerk „modulo 2^n“: Alles über 2^n fällt weg. −x und 2^n − x sind modulo 2^n dasselbe. Das Zweierkomplement von x ist genau 2^n − x (z. B. 256 − 38 = 218 = 1101 1010). Sonderfall für Prüfer: Das Zweierkomplement von −128 (1000 0000) ist wieder 1000 0000, weil +128 nicht darstellbar ist.

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Blatt 2 VII (nicht im Skript-Kapitel)

Hexadezimaldarstellung

Baut auf: Umrechnen: Horner-Schema und Wertigkeiten

Lernziele

- Ich kann ein Bitmuster in Viererblöcken in Hex umwandeln und zurück.
- Ich kenne A–F als 10–15.
- Ich kann eine 2-stellige Hexzahl dezimal ausrechnen.

Lange Bitmuster sind schwer zu lesen. Weil 16 = 2^4 ist, passen genau **4 Bit in eine Hex-Ziffer**. Für die Werte 10 bis 15 nimmt man die Buchstaben A bis F. Hexzahlen kennzeichnet man mit h am Ende (oder 0x davor).

Bits	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7
Bits	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	A (10)	B (11)	C (12)	D (13)	E (14)	F (15)

Binär → Hex: Von rechts in Viererblöcke teilen und jeden Block mit 8-4-2-1 umrechnen.

`1001 1100` → `1001` = 8+1 = 9, `1100` = 8+4 = 12 = C → **`9Ch`**

Hex → binär: Jede Ziffer wird zu 4 Bit. 7Ah → 7 = 0111, A = 10 = 1010 → `0111 1010`

Hex → dezimal: 7Ah = 7·16 + 10 = 112 + 10 = 122

Ergänzung · Hex im Alltag

Merke dir diese Werte, sie tauchen ständig auf: **FF = 255** (1111 1111), **80 = 128** (1000 0000), **7F = 127** (0111 1111), **10 = 16**.

Ein Farbwert wie `#FF8000` sind drei Bytes: Rot FF = 255, Grün 80 = 128, Blau 00 = 0. Auch MAC-Adressen und Speicheradressen werden in Hex geschrieben, weil ein Byte genau zwei Hex-Ziffern hat.

Beispiel: `1111 0000` → F = 1111, 0 = 0000 → **F0h** = 15·16 + 0 = 240.

Typische Fehler

- Von links statt von rechts in Vierergruppen geteilt (bei Längen, die kein Vielfaches von 4 sind).
- A = 11 statt 10.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Warum nicht Oktal (Basis 8, 3 Bit pro Ziffer)? Weil Bytes 8 Bit haben – 8 ist durch 4 teilbar, aber nicht durch 3. Ein Byte ist also genau zwei Hexziffern. Schreibweisen: 9Ch, 0x9C, $9C oder $9C_{16}$ meinen dasselbe.

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Skript S. 46–50 · Blatt 2 I–III

Halbaddierer, Volladdierer, Addierwerk

Baut auf: Bits, Wertigkeit, MSB und LSB

Lernziele

- Ich kann Wahrheitstabelle und Gleichungen des Halbaddierers angeben.
- Ich kann die Wahrheitstabelle des Volladdierers angeben und erklären, warum c_in nötig ist.
- Ich kann einen VA aus zwei HA bauen.
- Ich kann ein n-Bit-Addierwerk mit CF und ZF zeichnen.

Halbaddierer (HA)

Er addiert zwei Bits a und b. Das Ergebnis kann höchstens 1 + 1 = 2 = 10_2 sein, deshalb braucht man 2 Ausgänge: das Ergebnis e und den Übertrag c

(carry).

b	a	c	e
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

e = a ⊕ b (XOR, „=1“) · c = a & b (AND)

Volladdierer (VA)

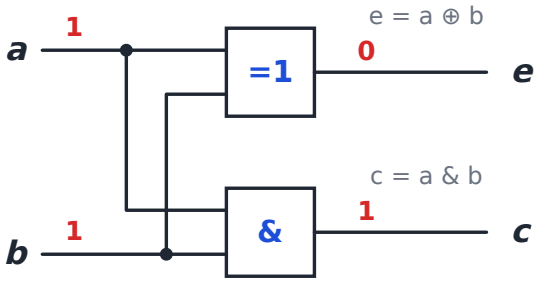
Ab der zweiten Stelle muss man den Übertrag c_{in} der Stelle davor mitzählen. Das sind drei Eingänge, die Summe ist höchstens 1+1+1 = 3 = 11_2. Zwei Ausgänge (e und c_{out}) reichen also noch.

c_{in}	b	a	c_{out}	e	Summe
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	1	0	2 = 10_2
1	0	0	0	1	1
1	0	1	1	0	2 = 10_2
1	1	0	1	0	2 = 10_2
1	1	1	1	1	3 = 11_2

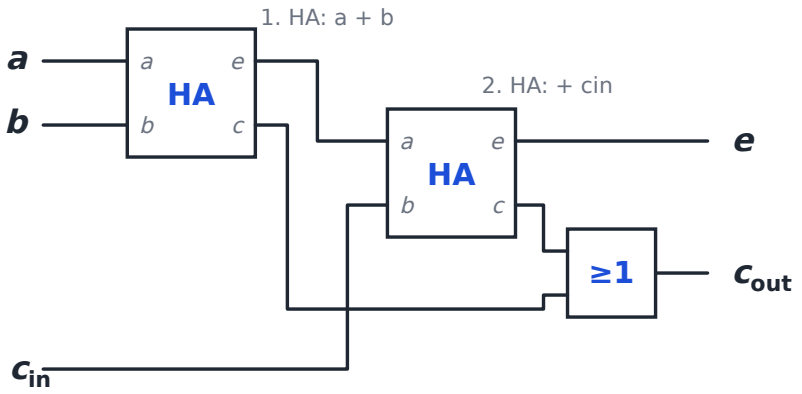
- Vereinfacht mit KV-Diagramm: e = a ⊕ b ⊕ c_{in} und c_{out} = (a&b) ∨ (a&c_{in}) ∨ (b&c_{in}). Zum zweiten Ausdruck: Der Übertrag ist 1, sobald mindestens zwei Eingänge 1 sind.
- Man kann einen VA auch aus **2 Halbaddierern + OR** bauen. Der 1. HA rechnet a + b, der 2. HA zählt c_{in} dazu. Die beiden Überträge werden mit OR verknüpft. Sie können nie beide 1 sein, deshalb reicht OR.

Addierwerk (n Bit)

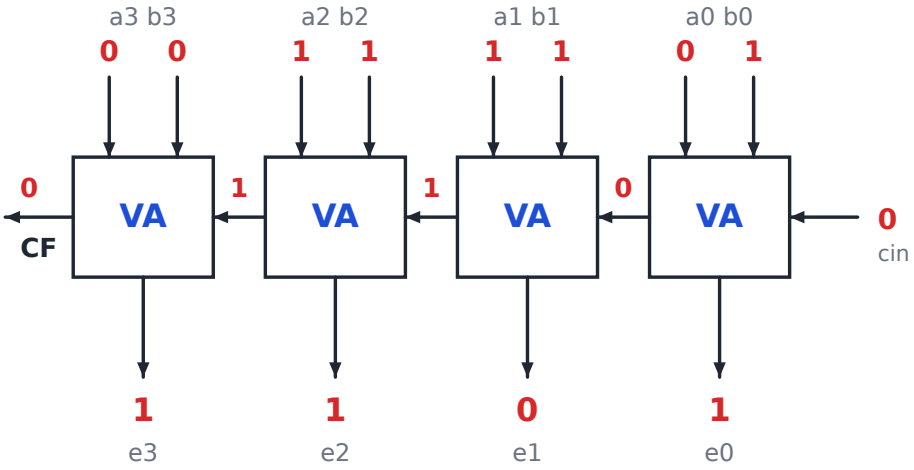
Man schaltet n Volladdierer hintereinander: c_{out} jeder Stelle geht an c_{in} der nächsthöheren Stelle. Beim LSB ist c_{in} = 0. Der letzte c_{out} ist das **Carry**. Die Schaltung mit Zero-Flag ist bei Blatt 2 · III gezeichnet.



Halbaddierer mit Beispiel 1 + 1 = 10_2: Ergebnis e = 0, Übertrag c = 1. Das XOR liefert das Ergebnis, das AND den Übertrag.



Volladdierer aus zwei Halbaddierern und einem OR. Der erste Halbaddierer rechnet a + b, der zweite zählt den Übertrag cin dazu.



4-Bit-Addierwerk, Beispiel 0110 + 0111 = 1101 (6 + 7 = 13). Die roten Zahlen sind die Signalwerte. Der Übertrag wandert von rechts nach links.

Ergänzung · Die Tabellen in Worten

- **Halbaddierer:** Das Ergebnis e ist 1, wenn genau eines der Bits 1 ist. Der Übertrag c ist nur 1, wenn beide 1 sind.
- **Volladdierer:** Das Ergebnis e ist 1 bei einer *ungeraden* Anzahl von Einsen unter den drei Eingängen (1 oder 3). Der Übertrag cout ist 1, sobald *mindestens zwei* Eingänge 1 sind.

Beispiel in der Zeichnung, Bit für Bit von rechts:

Bit	a	b	cin	e	cout
0	0	1	0	1	0
1	1	1	0	0	1
2	1	1	1	1	1
3	0	0	1	1	0

Ergebnis von oben nach unten gelesen: e3 e2 e1 e0 = 1101 = 13, und der letzte Übertrag ist 0.

Typische Fehler

- Beim HA das Ergebnis mit AND statt XOR gebildet.
- c_in des ersten VA nicht auf 0 gesetzt.
- Zero-Flag mit OR statt NOR gebildet.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Das Addierwerk aus hintereinander geschalteten Volladdierern heißt *Ripple-Carry-Addierer*: Der Übertrag „rieselt“ von rechts nach links. Bei 64 Bit muss er im schlimmsten Fall durch 64 Volladdierer laufen – das ist langsam. Echte Prozessoren nutzen deshalb Carry-Lookahead-Addierer, die Überträge parallel vorausberechnen (Hintergrundwissen, nicht im Skript).

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Skript S. 50–52

Subtraktion als Ersatzaddition

Baut auf: Zweierkomplement bilden, Halbaddierer, Volladdierer, Addierwerk

Lernziele

- Ich kann a − b als a + (−b) mit einem Addierwerk rechnen.
- Ich kann erklären, wofür das XOR und c_in = 1 im Rechenwerk sorgen.

Was wollen wir? a − b rechnen, aber kein eigenes Subtrahierwerk bauen.
Werkzeug: a − b = a + (−b). Den Vorzeichenwechsel liefert das Zweierkomplement.
Wie genau? Die Bits von b invertieren und c_{in} des ersten VA auf 1 setzen. Diese 1 ist genau das „+1“.

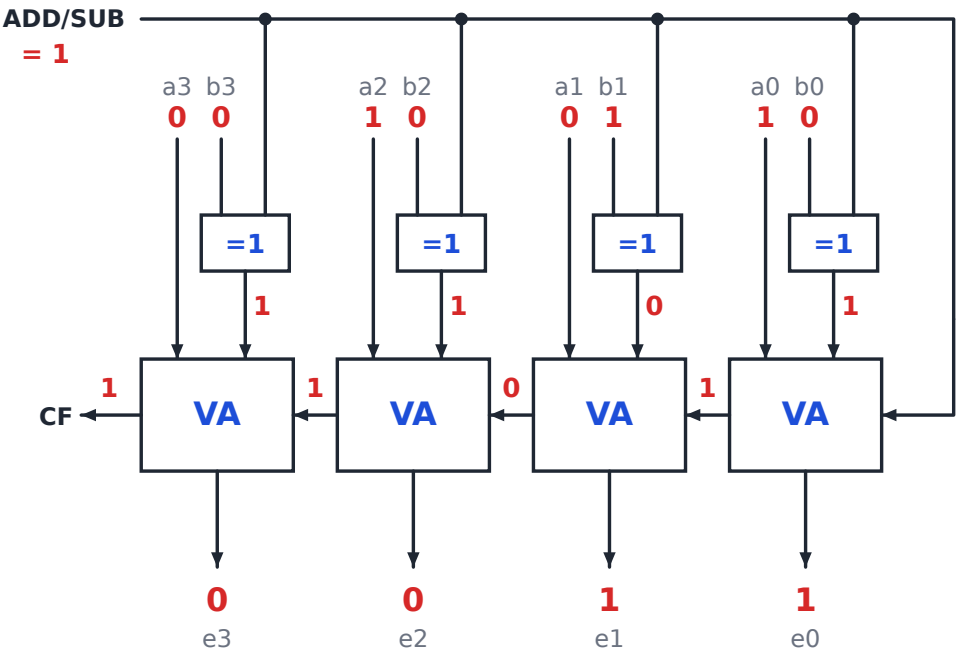
Beispiel: 0101 − 0010 (5 − 2)

```
  0101     (a = 5)
  1101     (b = 0010 bitweise invertiert)
+    1     (c_in = 1)
  ————
1 0011     = 3 ✓   (Carry 1 → bei Subtraktion heißt das: alles in Ordnung, siehe A8)
```

Rechenwerk für ADD und SUB: Vor jedem b-Eingang sitzt ein XOR. Die Steuerleitung `ADD/SUB` geht an alle XOR-Gatter und zusätzlich an c_{in}.

- ADD/SUB = 0: Das XOR lässt b unverändert durch ($b \oplus 0 = b$), und $c_{in} = 0$. Das Werk **addiert**.
- ADD/SUB = 1: Das XOR invertiert b ($b \oplus 1 = \bar{b}$), und $c_{in} = 1$. Das Werk **subtrahiert**.

Warum ein XOR als Schalter?
$x \oplus 0 = x$ und $x \oplus 1 = \bar{x}$. Ein XOR ist also ein „Invertierer zum Einschalten“: Die Steuerleitung entscheidet, ob invertiert wird.



Rechenwerk für ADD und SUB, Beispiel 5 − 2 = 3. Bei ADD/SUB = 1 invertieren die XOR-Gatter b, und der Übertrag cin des ersten Volladdierers ist 1.

Ergänzung · Das Beispiel in der Schaltung lesen

In der Zeichnung ist ADD/SUB = 1, das Werk subtrahiert. Aus b = 0010 macht das XOR $\bar{b}$ = 1101, und der erste Volladdierer bekommt cin = 1. Gerechnet wird also 0101 + 1101 + 1 = 1 0011. Die vier unteren Bits 0011 sind das Ergebnis 3, der Übertrag CF = 1 heißt bei der Subtraktion: alles in Ordnung (siehe nächstes Thema).

Merke: Ein einziges Steuersignal reicht, weil das XOR als „Schalter zum Invertieren“ arbeitet und dasselbe Signal gleichzeitig das +1 liefert.

Typische Fehler

- b invertiert, aber c_in auf 0 gelassen (ergibt a − b − 1).
- Bei der Subtraktion den Carry wie bei der Addition interpretiert.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Warum genügt ein XOR pro Bit plus c_in? Weil invertieren + 1 genau das Zweierkomplement ist. Das „+1“ kostet keinen eigenen Addierer, weil der Übertragseingang des ersten VA ohnehin frei ist. Nachfrage: „Was bedeutet der Übertrag bei der Ersatzaddition?“ – Er ist das Gegenteil des Borrow: Übertrag 1 = kein Borgen nötig = vorzeichenlos korrekt.

Zahlendarstellung und Rechnen Relevanz hoch Quelle: Skript S. 52–58 · Blatt 2 V–VI

Carry, Overflow und Zero-Flag

Baut auf: Zweierkomplement bilden, Halbaddierer, Volladdierer, Addierwerk, Subtraktion als Ersatzaddition

Lernziele

- Ich kann CF bestimmen und weiß: CF = 1 → vorzeichenlos falsch.
- Ich kann OF bestimmen (Vorzeichenregel und XOR der letzten zwei Überträge).
- Ich kann ZF bestimmen und weiß, wie es geschaltet wird (NOR).
- Ich kenne die Tabelle CF/OF → korrekt/falsch.

Das Addierwerk rechnet die Bits immer gleich. Ob das Ergebnis *stimmt*, hängt davon ab, wie *wir* die Bits lesen. Dazu gibt es die Flags.

Carry-Flag (CF): Fehler bei vorzeichenloser Rechnung

CF ist der Übertrag aus dem letzten (höchsten) Volladdierer. CF = 1 heißt: Das Ergebnis braucht ein Bit mehr, als vorhanden ist.

```
  1111 1111   vorzeichenlos 255 | Zweierkomplement −1
+ 0000 0001                   1 |                   1
 1————————
  0000 0000                  0 ✗ |                  0 ✓
  CF = 1
```

Man sieht: Carry ist **kein** Fehler im Zweierkomplement. Dort ist −1 + 1 = 0 richtig.

Overflow-Flag (OF): Fehler im Zweierkomplement

Ein Overflow liegt vor, wenn **zwei Summanden mit gleichem Vorzeichen** ein Ergebnis mit **anderem Vorzeichen** ergeben:

- positiv + positiv ergibt negativ → falsch
- negativ + negativ ergibt positiv → falsch
- Bei unterschiedlichen Vorzeichen kann nie ein Overflow entstehen.

```
  0110 0001   (97)
+ 0011 0000   (48)
  ————————
  1001 0001   → im Zweierkomplement −111 ✗  OF = 1, CF = 0
              (97 + 48 = 145 > 127, passt nicht)
```

```
OF = (Übertrag IN den letzten VA) ⊕ (Übertrag AUS dem letzten VA)
```

Merksatz: Overflow gibt es, wenn ein Übertrag in den letzten Volladdierer *hinein* **oder** aus ihm *heraus* geschoben wird, aber nicht, wenn er „durch ihn durch“ geht. Die vier Fälle aus dem Skriptum:

Fall	Rechnung	Übertrag hinein	Übertrag heraus	OF
1 · kein Übertrag	80 + (−112) = −32	0	0	0 ✓
2 · nur hinein	80 + 81 → −95	1	0	1 ✗
3 · nur heraus	−112 + (−111) → 33	0	1	1 ✗
4 · durch	−48 + (−79) = −127	1	1	0 ✓

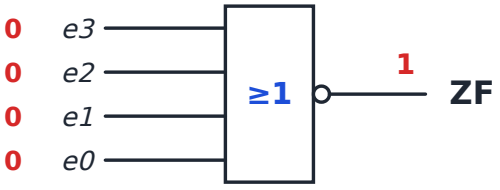
CF	OF	vorzeichenlos	Zweierkomplement
0	0	korrekt	korrekt
1	0	falsch	korrekt
0	1	korrekt	falsch
1	1	falsch	falsch

Carry bei der Subtraktion

Bei der Ersatzaddition bedeutet der Übertrag das *Gegenteil* einer „geborgten“ Stelle. Beispiel: 9 − 7 mit 8 Bit ergibt 0000 0010 = 2, und das stimmt, **obwohl** der Übertrag 1 ist. Deshalb wird der Carry bei der Subtraktion vor der Weiterverarbeitung **invertiert**. Danach gilt immer: **CF = 1 heißt Fehler**, egal ob addiert oder subtrahiert wurde. Beim Overflow braucht man nichts zu ändern.

Zero-Flag (ZF), kommt in Blatt 2 vor

ZF = 1 genau dann, wenn das Ergebnis 0000…0 ist. In der Schaltung gehen alle Ergebnisbits in ein OR mit negiertem Ausgang, also ein **NOR**. Ist irgendein Bit 1, liefert das OR eine 1 und die Negation macht daraus ZF = 0.



NOR: nur bei lauter Nullen ist ZF = 1

Zero-Flag für 4 Bit: ein NOR über alle Ergebnisbits. Hier ist das Ergebnis 0000, also ZF = 1.

Ergänzung · Welches Flag lese ich?

Das Addierwerk weiß nicht, ob du die Bits vorzeichenlos oder im Zweierkomplement liest. Es setzt beide Flags, und **du** entscheidest, welches zählt:

- Du rechnest **vorzeichenlos** → CF prüfen (CF = 1 heißt Fehler).
- Du rechnest im **Zweierkomplement** → OF prüfen (OF = 1 heißt Fehler).

Beispiel 8 Bit: 1100 1000 + 0110 0100

```
  1100 1000   (200 vorzeichenlos | −56 im Zweierkomplement)
+ 0110 0100   (100 vorzeichenlos | +100)
1 0010 1100   → 0010 1100 = 44
```

Vorzeichenlos: 200 + 100 = 300 passt nicht in 8 Bit, also **CF = 1**, das Ergebnis 44 ist falsch. Im Zweierkomplement: −56 + 100 = 44 stimmt. Die Vorzeichen waren verschieden, also **OF = 0**.

Typische Fehler

- Carry als Fehlerindikator im Zweierkomplement verwendet.
- Overflow bei unterschiedlichen Vorzeichen der Summanden vermutet (kann nie passieren).
- Bei Subtraktion vergessen, dass der Übertrag invertiert wird.

Zum Vertiefen: Hintergründe und Prüfernachfragen

In echten Prozessoren liegen diese Flags im Statusregister (oft dazu noch ein Sign/Negative-Flag = MSB des Ergebnisses). Das Carry wird auch für Mehrwort-Arithmetik genutzt: Man addiert zuerst die unteren 8 Bit, dann die oberen 8 Bit plus Carry – so kann ein 8-Bit-Prozessor 16-Bit-Zahlen addieren. Nachfrage: „Warum ist OF = c_in(MSB) ⊕ c_out(MSB)?“ → Ein Übertrag nur hinein dreht das Vorzeichen fälschlich auf 1, nur heraus fälschlich auf 0; bei beiden heben sie sich auf.

Zahlendarstellung und Rechnen Relevanz mittel Quelle: Skript S. 58–60

Bitstellen hinzufügen (Vorzeichenerweiterung)

Baut auf: Die vier Darstellungen im Vergleich, Zweierkomplement bilden

Lernziele

- Ich kann eine 4-Bit-Zahl in allen vier Darstellungen auf 8 Bit erweitern.
- Ich kann die Vorzeichenerweiterung beim Zweierkomplement begründen.

Darstellung	Regel	Beispiel 4 Bit	8 Bit
vorzeichenlos	vorne Nullen auffüllen	`1011 (11)`	`0000 1011`
Vorzeichen-Betrag	Vorzeichen ins neue MSB, dazwischen Nullen	`1101 (−5)`	`1000 0101`
Exzess	die Differenz der beiden Exzesse addieren (127 − 7 = 120)	`0010 (−5)`	`0000 0010 + 0111 1000 = 0111 1010`
Zweierkomplement	**Vorzeichenerweiterung**: das MSB in alle neuen Stellen kopieren	`1101 (−3)`	`1111 1101`

Probe für −3: −128 + 64 + 32 + 16 + 8 + 4 + 1 = −3 ✓

Warum lauter Einsen bei negativen Zahlen?
Bei 4 Bit hat das MSB die Wertigkeit −8, bei 8 Bit −128. Damit wieder derselbe Wert herauskommt, müssen die neuen Bits die Differenz ausgleichen: −128 + 64 + 32 + 16 + 8 = −8 ✓

Ergänzung · Ein Muster, zwei Erweiterungen

Das 4-Bit-Muster 1011 wird je nach Lesart *unterschiedlich* auf 8 Bit erweitert:

Lesart	4 Bit	8 Bit	Wert
vorzeichenlos	1011	0000 1011	11
Zweierkomplement	1011	1111 1011	−5

Probe für −5: −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5 ✓. Darum immer zuerst fragen: **Welche Darstellung?** Erst dann erweitern.

Typische Fehler

- Negative Zweierkomplement-Zahlen mit Nullen erweitert (macht aus −3 plötzlich +13).
- Bei Exzess nur Nullen ergänzt, statt den Exzess-Unterschied zu addieren.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Vorzeichenerweiterung (sign extension) passiert in jedem Prozessor, wenn ein Byte in ein 32- oder 64-Bit-Register geladen wird. In C entscheidet der Datentyp: signed char wird mit MSB-Kopien erweitert, unsigned char mit Nullen. Genau das ist eine häufige Fehlerquelle beim Programmieren.

Zahlendarstellung und Rechnen · Relevanz mittel · Quelle: Skript S. 60–61

Festkommadarstellung

Baut auf: Umrechnen: Horner-Schema und Wertigkeiten

Lernziele

- Ich kann eine Festkommazahl über die Wertigkeiten 2^{-1}, 2^{-2}, … lesen.
- Ich kann den Nachkommateil durch wiederholtes Multiplizieren mit 2 umrechnen.
- Ich kann erklären, warum Rundungsfehler entstehen (z. B. 0,2).

Das Komma sitzt an einer festen Stelle. Die Stellen dahinter haben die Wertigkeiten $2^{-1} = 0{,}5$, $2^{-2} = 0{,}25$, $2^{-3} = 0{,}125$ und so weiter.

2^4	2^3	2^2	2^1	2^0	,	2^{-1}	2^{-2}	2^{-3}
1	0	1	1	0	,	1	0	1

$10110{,}101_2 = 16 + 4 + 2 + 0{,}5 + 0{,}125 = 22{,}625$

Dezimal → Festkomma am Beispiel 14,6875: Den ganzzahligen Teil rechnet man mit Horner um, 14 = 1110. Den Nachkommateil multipliziert man immer wieder mit 2. Die Stelle vor dem Komma ist jeweils das nächste Bit. Man nimmt sie weg und rechnet mit dem Rest weiter. Diese Bits liest man **von oben nach unten**.

0,6875 · 2 = 1,375	Bit 1 (dann mit 0,375 weiter)
0,375 · 2 = 0,75	Bit 0
0,75 · 2 = 1,5	Bit 1 (dann mit 0,5 weiter)
0,5 · 2 = 1,0	Bit 1, Rest 0, fertig

Ergebnis: **$14{,}6875 = 1110{,}1011_2$**. Probe: 8+4+2 + 0,5+0,125+0,0625 = 14,6875 ✓

Rundungsfehler: 0,2 ergibt 0,0011 0011 0011…$_2$ und hat damit unendlich viele Stellen. Mit endlich vielen Bits muss man runden. Addiert und subtrahiert wird mit demselben Addierwerk wie bisher.

Ergänzung · Festkomma ist eine ganze Zahl mit gedachtem Komma

Man kann $10110{,}101_2$ auch als ganze Zahl lesen und durch 2^3 teilen: $1011\ 0101_2 = 181$, und 181 : 8 = 22,625 ✓. Das Komma ist nur gedacht. Deshalb rechnet das Addierwerk mit Festkommazahlen genauso wie mit ganzen Zahlen, solange alle Zahlen dieselbe Kommastelle haben.

Zum Abschätzen: $0{,}1_2 = 0{,}5 \cdot 0{,}01_2 = 0{,}25 \cdot 0{,}001_2 = 0{,}125 \cdot 0{,}0001_2 = 0{,}0625$.

Typische Fehler

- Nachkommabits von unten nach oben gelesen (das ist nur beim Horner-Schema so).
- Wertigkeit 2^{-2} als 0,5 statt 0,25.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Eine Dezimalzahl hat im Binärsystem nur dann endlich viele Nachkommastellen, wenn ihr gekürzter Bruch im Nenner nur Zweierpotenzen hat: 0,375 = 3/8 ✓, aber 0,2 = 1/5 ✗ (Faktor 5). Gleitkomma (IEEE 754) verschiebt das Komma je nach Zahl – Festkomma nicht; darum ist Festkomma einfacher, hat aber einen kleinen Wertebereich.

Zahlendarstellung und Rechnen · Relevanz niedrig · Quelle: Skript S. 61–62 · NIST

Binärpräfixe (kibi, mebi, gibi)

Baut auf: Bits, Wertigkeit, MSB und LSB

Lernziele

- Ich kenne die Binärpräfixe Ki, Mi, Gi, Ti, Pi, Ei und ihre Werte.
- Ich kann den Unterschied zwischen kB und KiB berechnen.

Zehnerpotenz	Wert	Zweierpotenz	Wert
kilo (k)	$10^3 = 1\,000$	kibi (Ki)	$2^{10} = 1\,024$
Mega (M)	10^6	mebi (Mi)	$2^{20} = 1\,048\,576$
Giga (G)	10^9	gibi (Gi)	2^{30}
Tera (T)	10^{12}	tebi (Ti)	2^{40}
Peta (P)	10^{15}	pebi (Pi)	2^{50}
Exa (E)	10^{18}	exbi (Ei)	2^{60}

Der Unterschied wird mit der Größe immer größer: Zwischen kB und KiB sind es 2,4 % (24 Byte), zwischen TB und TiB schon fast 10 %. Deshalb wirkt eine Festplatte im Betriebssystem kleiner als auf der Packung.

Ergänzung · Rechenbeispiele

- **Festplatte:** 1 TB sind 10^{12} Byte. Das sind $10^{12} : 2^{30} \approx 931$ GiB, also etwa 0,91 TiB. Das erklärt, warum die Platte im Betriebssystem kleiner aussieht.
- **Arbeitsspeicher:** 4 GiB = $4 \cdot 2^{30} = 4\,294\,967\,296$ Byte.
- **Adressleitungen:** 16 KiB = 2^{14} Byte ($16 \cdot 2^{10}$). Bei einem byteweise adressierten Speicher braucht man dafür 14 Adressleitungen.

Typische Fehler

- KiB und kB gleichgesetzt.
- Unterschied für alle Größen als 2,4 % angenommen (wächst mit der Größe).

Zum Vertiefen: Hintergründe und Prüfernachfragen

Die IEC hat die Binärpräfixe im Dezember 1998 beschlossen und 1999 als Ergänzung zur IEC 60027-2 veröffentlicht. Vorher meinten Informatiker mit „kilo“ oft 1024, weil $2^{10} \approx 1000$ ist. Heute: kB = 1000 Byte, KiB = 1024 Byte.

Schaltwerke und Speicher

Schaltnetze haben kein Gedächtnis. Erst durch Rückkopplung entstehen Speicher: erst das RS-Flip-Flop, daraus Latch und Flip-Flop, daraus Register, Zähler und ganze Schaltwerke. In diesem Teil sind die Schaltbilder besonders wichtig: Wenn du sie lesen und selbst zeichnen kannst, hast du den Stoff verstanden.

Roter Faden: Speicher entsteht durch Rückkopplung. Ein Takt sorgt dafür, dass sich alles nur zu festen Zeitpunkten ändert.

Schaltwerke und Speicher Relevanz hoch Quelle: Skript S. 72–74

Schaltnetz vs. Schaltwerk, RS-Flip-Flop

Lernziele

- Ich kann Schaltnetz und Schaltwerk unterscheiden (Rückkopplung, Speicher).
- Ich kenne die Funktionstabelle des RS-Flip-Flops.
- Ich kann begründen, warum S = R = 1 verboten ist.
- Ich kenne Anwendungen (Ein/Aus-Taster, Entprellen).

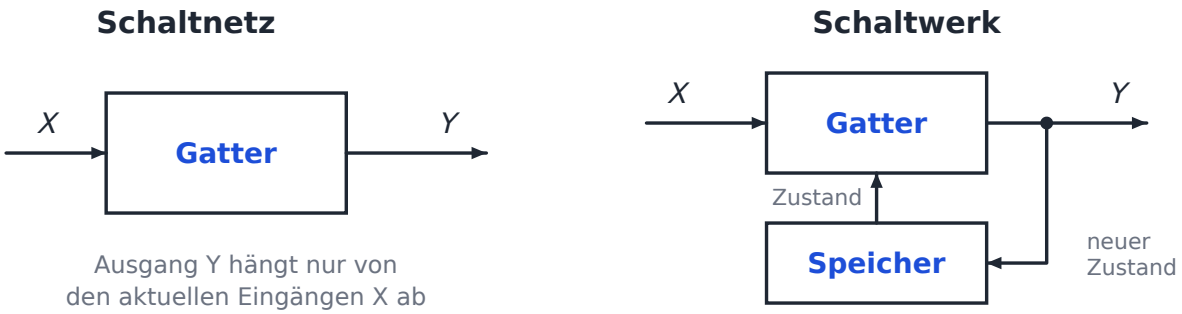
- **Schaltnetz:** Der Ausgang hängt *nur* von den aktuellen Eingängen ab. Es gibt keine Rückkopplung und keinen Speicher. Beispiele sind Addierer, Decoder und Multiplexer.
- **Schaltwerk:** Durch Rückkopplung (ein Ausgang geht zurück auf einen Eingang) und durch Gatterlaufzeiten können Zustände **gespeichert** werden. Der Ausgang hängt dann auch von der Vergangenheit ab.

Es besteht aus zwei NOR-Gattern, deren Ausgänge über Kreuz auf den jeweils anderen Eingang zurückgeführt sind.

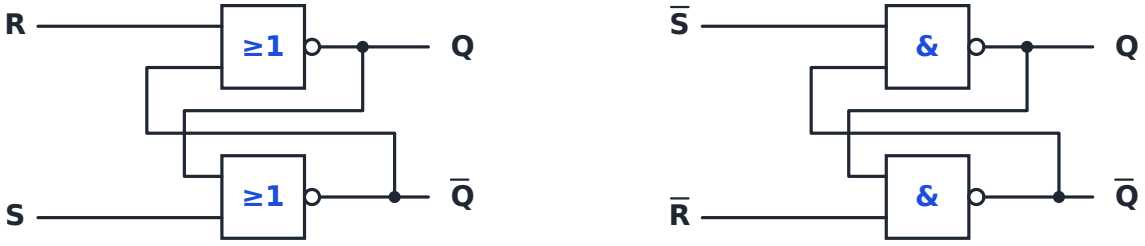
S	R	Wirkung
0	0	speichern (Q bleibt)
1	0	setzen → Q = 1
0	1	rücksetzen → Q = 0
1	1	verboten: Q = $\bar{Q}$ = 0. Gehen danach beide auf 0, ist der Zustand unvorhersehbar und das Flip-Flop kann schwingen

Warum S = 1 setzt: S = 1 erzwingt $\bar{Q}$ = 0 (ein NOR mit einer 1 am Eingang liefert immer 0). Dann sind am oberen NOR R = 0 und $\bar{Q}$ = 0, also wird Q = `NOT(0 v 0)` = 1.

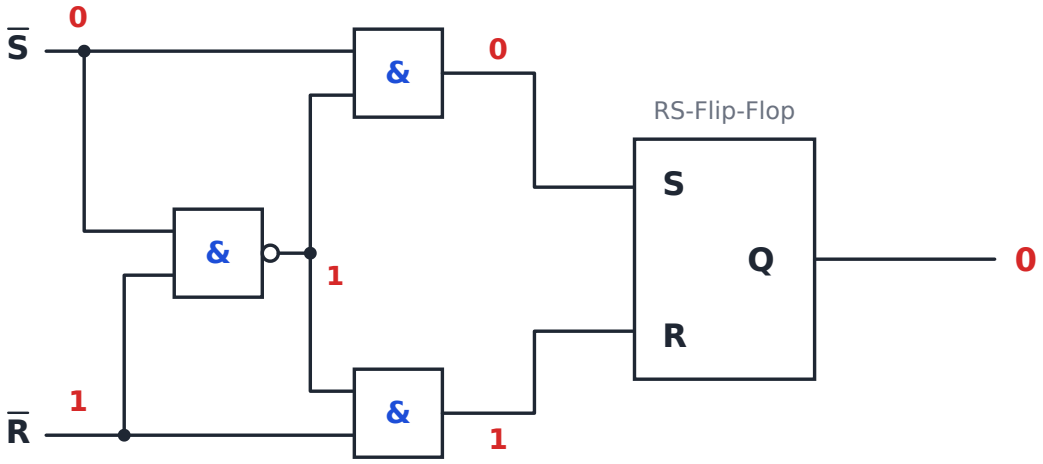
- Aus **NAND-Gattern** gebaut hat das Flip-Flop **0-aktive** Eingänge, im Symbol erkennt man sie an den Negationsringen.
- Anwendungen: Ein/Aus mit zwei Tastern (mit Pull-up-Widerständen) und das **Entprellen** eines Schalters. Das Flip-Flop schaltet bei der ersten Kontaktberührung und ignoriert das Nachfedern danach.



Schaltnetz und Schaltwerk im Vergleich: Das Schaltwerk hat zusätzlich einen Speicher, dessen Inhalt auf die Gatter zurückwirkt.



Links: RS-Flip-Flop aus zwei NOR-Gattern (Eingänge 1-aktiv). Rechts: aus zwei NAND-Gattern (Eingänge 0-aktiv, erkennbar am Überstrich).



Beispiel aus dem Unterricht: Vorschaltung vor dem RS-Flip-Flop. Rot sind die Signalwerte für S̄ = 0 und R̄ = 1.

Ergänzung · Wie ein RS-Flip-Flop speichert

Im NOR-Flip-Flop speist jeder Ausgang den anderen Eingang. Dadurch hält sich ein Zustand selbst. Ablauf von Q = 0 aus:

Schritt	S	R	Q danach	Warum
1	1	0	1	setzen
2	0	0	1	speichern: Q bleibt 1, obwohl S wieder 0 ist
3	0	1	0	rücksetzen
4	0	0	0	speichern: Q bleibt 0

Schritt 2 und 4 sind der eigentliche Speicher: Bei gleichen Eingängen (0, 0) ist der Ausgang unterschiedlich, je nachdem, was vorher war. Ein Schaltnetz kann das nie.

NAND-Variante: Die Eingänge sind 0-aktiv. $\bar{S} = 0$ setzt (Q = 1), $\bar{R} = 0$ setzt zurück, $\bar{S} = \bar{R} = 1$ speichert, $\bar{S} = \bar{R} = 0$ ist verboten (dann sind beide Ausgänge 1).

Ergänzung · Das Beispiel aus dem Unterricht

Gelesen wird die Skizze so (zur Sicherheit mit deinen Unterrichtsunterlagen vergleichen): Das kleine Gatter in der Mitte ist ein NAND der beiden Eingänge $\bar{S}$ und $\bar{R}$. Sein Ausgang n geht an zwei UND-Gatter, die jeweils noch einen der Eingänge bekommen. Die Ausgänge der UND-Gatter sind S und R des RS-Flip-Flops.

$$n = \text{NAND}(\bar{S}, \bar{R}) \cdot S = \bar{S} \,\&\, n \cdot R = \bar{R} \,\&\, n$$

$\bar{S}$	$\bar{R}$	n	S	R	Wirkung auf Q
0	0	1	0	0	speichern
0	1	1	0	1	rücksetzen → Q = 0 (Beispiel in der Skizze)
1	0	1	1	0	setzen → Q = 1
1	1	0	0	0	speichern

Was die Vorschaltung bringt: Sind $\bar{S}$ und $\bar{R}$ beide 1, sperrt das NAND (n = 0) beide UND-Gatter. Dadurch erreicht das RS-Flip-Flop den verbotenen Zustand S = R = 1 **nie**. Bei gleichen Eingängen wird gespeichert, bei verschiedenen Eingängen gesetzt oder rückgesetzt.

Nachrechnen der roten Werte: $\bar{S} = 0$, $\bar{R} = 1$ → n = NAND(0, 1) = 1. Oben: 0 & 1 = 0, unten: 1 & 1 = 1. Also S = 0, R = 1 und damit Q = 0.

Typische Fehler

- Bei NAND-Flip-Flops die 0-aktiven Eingänge übersehen.
- Bei S = R = 0 einen festen Wert erwartet – der alte Wert bleibt.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Der „verbotene“ Zustand ist an sich stabil (Q = $\bar{Q}$ = 0). Das Problem entsteht erst, wenn S und R gleichzeitig wieder auf 0 gehen: Dann schalten beide NOR-Ausgänge auf 1, das koppelt zurück, beide gehen auf 0 … das Flip-Flop schwingt, bis eine kleine Laufzeitdifferenz entscheidet (metastabiler Zustand). Das Ergebnis ist zufällig.

Schaltwerke und Speicher Relevanz hoch Quelle: Skript S. 74–75 · Ü4 Aufgabe 2

D-Latch und D-Flip-Flop

Baut auf: Schaltnetz vs. Schaltwerk, RS-Flip-Flop

Lernziele

- Ich weiß: Latch ist bei C = 1 transparent (Q folgt D).
- Ich weiß: D-Flip-Flop übernimmt D nur bei der steigenden Flanke.
- Ich kann das Master-Slave-Prinzip erklären.
- Ich kann ein Zeitdiagramm für Latch und Flip-Flop zeichnen.

D-Latch (taktzustandsgesteuert)

Eingänge: D (Data) und C (Control).

- Solange **C = 1**: Q = D. Das Latch ist **transparent** und macht jede Änderung von D mit.
- Bei **C = 0**: Der letzte Wert bleibt gespeichert.

Entscheidend ist der *Zustand* (Pegel) von C.

D-Flip-Flop (taktflankengesteuert)

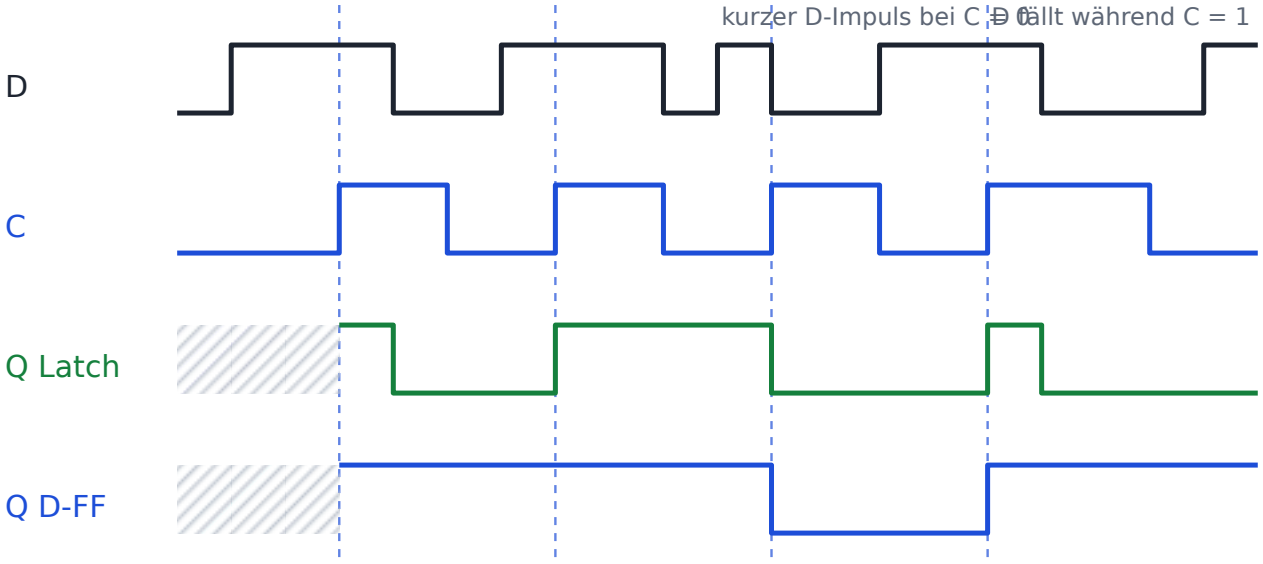
- Nur bei der **steigenden Taktflanke** (0 → 1) übernimmt Q den Wert von D.
- Sonst passiert nichts, auch wenn sich D ändert.

Es ist nach dem **Master-Slave-Prinzip** aufgebaut: zwei Latches hintereinander. Bei C = 0 ist der Master transparent, bei C = 1 der Slave. Weil immer nur eines offen ist, ist das Flip-Flop als Ganzes nie transparent.

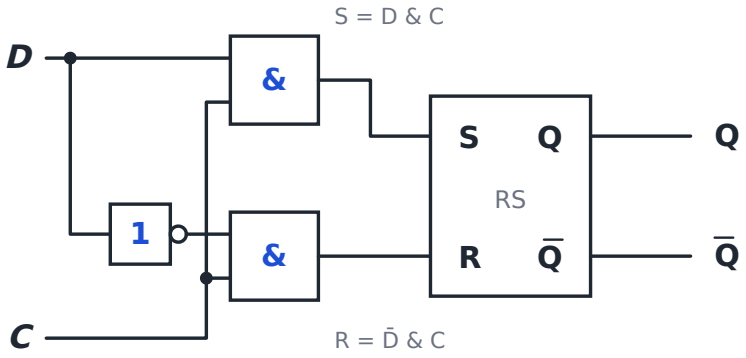
- Im Symbol bedeutet das **Dreieck** am Takteingang C1 „flankengesteuert“. Ein **Ring** davor bedeutet „schaltet bei der negativen Flanke“.
- Zusätzliche S- und R-Eingänge arbeiten **asynchron**, also unabhängig vom Takt.
- Bei Bausteinen mit mehreren Flip-Flops zeichnet man einen **Steuerkopf** für den gemeinsamen Takt und Reset.

Ein Zeitdiagramm-Beispiel mit beiden Bausteinen gibt es bei **Ü4 · 2**.

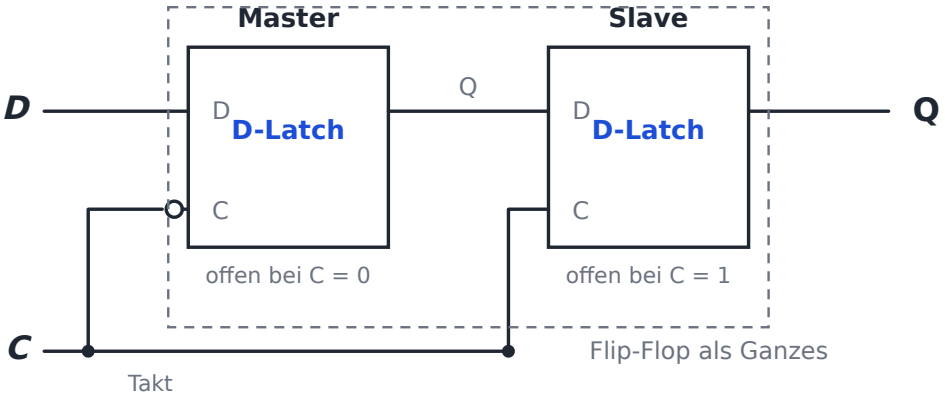
Für diese Aufgabe braucht man nur die zwei Regeln aus **B3**. Hier ein eigenes Beispiel mit beiden Bausteinen am selben Signal. Die gestrichelten Linien markieren die steigenden Flanken von C.



Schraffiert = Q ist unbekannt (vor dem ersten C = 1 bzw. vor der ersten steigenden Flanke).



D-Latch aus einem RS-Flip-Flop und drei Gattern. Weil S = D & C und R = D̄ & C nie gleichzeitig 1 sein können, gibt es keinen verbotenen Zustand.



D-Flip-Flop nach dem Master-Slave-Prinzip: zwei Latches, die abwechselnd offen sind.

Ergänzung · Latch oder Flip-Flop?

Latch: Wie eine Tür, die offen ist, solange C = 1. Alles, was in dieser Zeit an D passiert, kommt durch.

Flip-Flop: Wie ein Foto. Nur im Moment der steigenden Flanke wird D „fotografiert“, danach ist die Tür zu.

Beim Master-Slave-Aufbau erklärt sich das so: Bei C = 0 nimmt der Master D auf, der Slave ist zu. Bei der Flanke wechselt C auf 1, der Master schließt mit dem zuletzt gesehenen Wert, und der Slave öffnet und gibt genau diesen Wert aus. Nie sind beide gleichzeitig offen.

Typische Fehler

- Beim Flip-Flop Änderungen während C = 1 mitgezeichnet.
- Beim Latch die kurzen D-Spitzen während C = 1 vergessen.
- Anfangsbereich nicht als unbekannt markiert.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Prüfernachfrage: „Warum ist ein Master-Slave-Flip-Flop nie transparent?“ – Master und Slave werden mit gegensätzlichem Takt angesteuert. Ist der Master offen (C = 0), ist der Slave zu, und umgekehrt. Ein Signal kommt also nie in einem Zug vom Eingang zum Ausgang. Der Moment der Übernahme ist der Wechsel 0 → 1: Der Master schließt (merkt sich D), der Slave öffnet (gibt es aus).

Schaltwerke und Speicher Relevanz hoch Quelle: Skript S. 76–77, 93 · Ü4 Aufgabe 1

JK-Flip-Flop

Baut auf: D-Latch und D-Flip-Flop

Lernziele

- Ich kenne die JK-Funktionstabelle (00 halten, 10 setzen, 01 rücksetzen, 11 kippen).
- Ich kann ein JK-Flip-Flop aus einem D-Flip-Flop bauen: D = (J & $\bar{Q}$) ∨ ($\bar{K}$ & Q).
- Ich kenne die Vorbereitungstabelle (Q_alt → Q_neu → J, K).

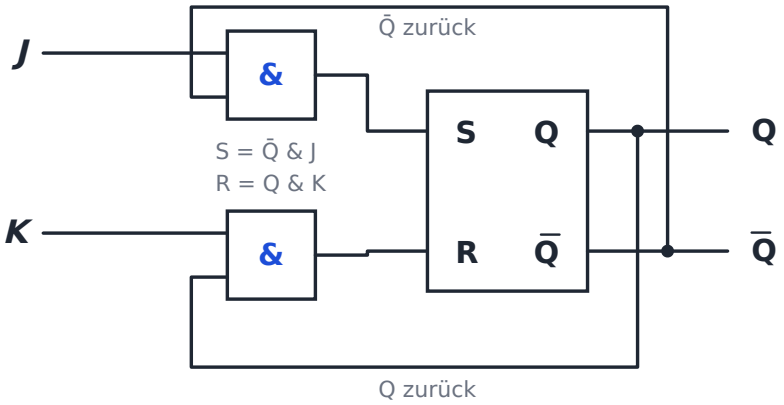
J	K	bei der Taktflanke
0	0	Q bleibt (store)
1	0	Q = 1 (set)
0	1	Q = 0 (reset)
1	1	Q kippt (toggle) → Grundlage für Zähler

Laut Skriptum baut man es aus einem RS-Flip-Flop mit Zusatzlogik: R = Q & K und S = $\bar{Q}$ & J, plus Master-Slave für die Flankensteuerung. Wie man ein JK-Flip-Flop aus einem D-Flip-Flop baut, zeigt die Übung **Ü4 · 1**.

Vorbereitungstabelle

$Q_{alt} \rightarrow Q_{neu}$	J	K	J = K (zusammengeschaltet)
0 → 0	0	X	0
0 → 1	1	X	1
1 → 0	X	1	1
1 → 1	X	0	0

Regel: Bei $Q_{alt} = 0$ ist $J = Q_{neu}$ und $K = X$. Bei $Q_{alt} = 1$ ist $J = X$ und $K = \bar{Q}_{neu}$.



JK-Flip-Flop aus einem RS-Flip-Flop mit Rückkopplung (Takt und Master-Slave sind hier der Übersicht halber weggelassen).

Ergänzung · Warum J = K = 1 kein verbotener Zustand ist

Durch die Rückkopplung ist $S = \bar{Q} \,\&\, J$ und $R = Q \,\&\, K$. Q und $\bar{Q}$ sind immer verschieden, also kann nie gleichzeitig $Q = 1$ und $\bar{Q} = 1$ gelten. Damit sind S und R nie beide 1. Bei $J = K = 1$ sperrt der Ausgang automatisch den Eingang, der nichts ändern würde: Ist $Q = 1$, kann nur R wirken, und Q kippt auf 0. Ist $Q = 0$, kann nur S wirken, und Q kippt auf 1.

Frequenzteiler: Mit $J = K = 1$ wechselt Q bei jeder Taktflanke den Wert. Ein volles Q-Signal braucht daher zwei Taktperioden. Die Ausgangsfrequenz ist f/2, aus 8 MHz werden 4 MHz.

Typische Fehler

- $J = K = 1$ mit dem verbotenen Zustand des RS verwechselt.
- In der D-Formel Q statt $\bar{Q}$ beim J-Term.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Ein JK mit $J = K = T$ heißt T-Flip-Flop (Toggle). Mit $J = K = 1$ halbiert es die Taktfrequenz – Grundbaustein aller Dualzähler. „Universell“ heißt: Jedes andere Flip-Flop (D, T, RS) lässt sich aus einem JK bauen, z. B. D-FF: $J = D$, $K = \bar{D}$.

Zeitangaben und Taktgeneratoren

Baut auf: D-Latch und D-Flip-Flop

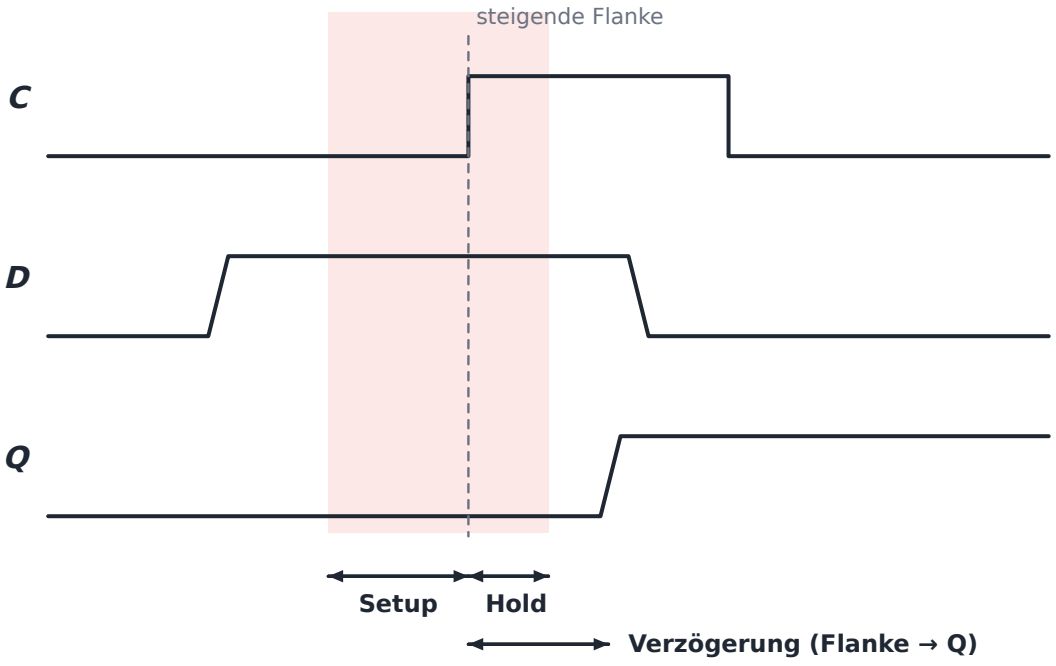
Lernziele

- Ich kann Setup-Zeit, Hold-Zeit und Propagation Delay erklären.
- Ich kenne drei Arten von Taktgeneratoren (Inverter + Verzögerung, RC + Schmitt-Trigger, Quarz).

- **Setup time:** So lange *vor* der Taktflanke muss D bzw. J/K schon stabil anliegen.
- **Hold time:** So lange *nach* der Flanke muss der Wert noch stabil bleiben.
- **Propagation delay:** Verzögerung von der Flanke bis zur Änderung des Ausgangs.
- Außerdem: maximale Taktfrequenz und minimale Dauer eines Taktimpulses. Beispiel 74LS74: Setup 20 ns, Hold 5 ns, maximal 33 MHz, Impulsdauer mindestens 25 ns, Verzögerung 20 ns.

Taktgeneratoren:

- Ein Inverter mit rückgekoppelter Verzögerung. Die Periode ist die doppelte Verzögerungszeit.
- **RC-Glied mit Schmitt-Trigger:** Der Kondensator lädt und entlädt sich zwischen den zwei Schaltschwellen. Ungenau.
- **Schwingquarz:** nutzt den Piezoeffekt und liefert eine sehr genaue Frequenz.



Zeitverhalten an der steigenden Taktflanke. Im rot markierten Bereich muss D stabil sein.

Ergänzung · Rechenbeispiele

Setup und Hold als Fenster: D darf sich im Zeitfenster rund um die Flanke (Setup davor, Hold danach) nicht ändern. Ändert es sich dort, ist der Ausgang nicht vorhersagbar.

Maximale Taktfrequenz (Beispielwerte, frei gewählt): Verzögerung des Flip-Flops t_1 = 20 ns, Gatterlogik t_2 = 30 ns, Setup-Zeit t_3 = 20 ns. Dann ist T_{min} = 20 + 30 + 20 = 70 ns und f_{max} = 1 / 70 ns ≈ 14,3 MHz.

Inverter-Oszillator: Bei 10 ns Verzögerung ist die Periode doppelt so lang, also 20 ns, und die Frequenz 50 MHz.

Typische Fehler

- Setup (vorher) und Hold (nachher) vertauscht.
- Taktperiode = Verzögerungszeit statt doppelte Verzögerungszeit.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Werden Setup- oder Hold-Zeit verletzt, kann ein Flip-Flop metastabil werden: Der Ausgang bleibt kurz zwischen 0 und 1 hängen und kippt dann zufällig. Darum synchronisiert man externe Signale (z. B. Taster) mit zwei hintereinander geschalteten Flip-Flops.

Schaltwerke und Speicher Relevanz mittel Quelle: Skript S. 80–81

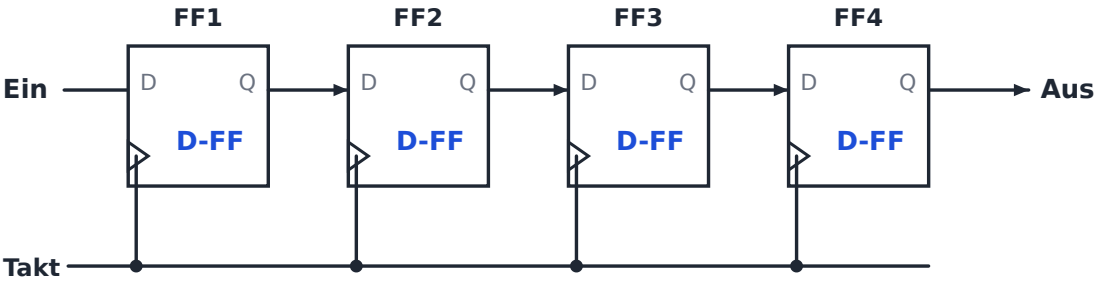
Register

Baut auf: D-Latch und D-Flip-Flop

Lernziele

- Ich kann ein Schieberegister und ein Parallelregister beschreiben.
- Ich kann begründen, warum das Schieberegister nur durch die FF-Verzögerung funktioniert.
- Ich kenne die Parallel-Serien-Umsetzung.

- **Schieberegister** (Serienregister): eine Kette aus D-Flip-Flops.
  - Bei jedem Takt rückt alles um eine Stelle weiter. Vorne kommt ein neues Bit hinein, das letzte geht verloren.
  - Das funktioniert nur, weil Flip-Flops eine Verzögerung haben. Sonst würden sich die D-Eingänge während der Flanke ändern.
  - Alle Flip-Flops müssen den Takt gleichzeitig bekommen. Wenn das nicht sicher ist, verwendet man Flip-Flops mit „Data-Lockout“.
  - Mit 2-zu-1-Multiplexern vor jedem D-Eingang kann man wahlweise nach **links oder rechts** schieben (Steuersignal R/$\bar{L}$).
- **Parallelregister:** übernimmt bei jedem Takt alle Bits gleichzeitig.
- Kombiniert man beide, erhält man eine **Parallel-Serien-Umsetzung** und umgekehrt. Bei P/$\bar{S}$ = 1 wird parallel geladen, bei P/$\bar{S}$ = 0 geschoben.



bei jeder steigenden Taktflanke rückt alles um eine Stelle nach rechts

Schieberegister aus vier D-Flip-Flops. Alle haben denselben Takt.

Ergänzung · Beispiel: Bits einschieben

Das Register startet mit 0000 (Q1 bis Q4). Am Eingang liegen nacheinander die Bits 1, 0, 1, 1. Bei jeder steigenden Flanke wandert alles eine Stelle nach rechts:

Takt	Eingang	Q1	Q2	Q3	Q4
Start	–	0	0	0	0
1	1	1	0	0	0
2	0	0	1	0	0
3	1	1	0	1	0
4	1	1	1	0	1

Das zuerst eingeschobene Bit steht nach vier Takten ganz rechts (Q4). Ein fünfter Takt würde es herausschieben.

Typische Fehler

- Schieberichtung verwechselt (FF1 bekommt den neuen Wert, FF4 verliert seinen).

Zum Vertiefen: Hintergründe und Prüfernachfragen

Serielle Schnittstellen (UART, SPI, USB) beruhen auf genau dieser Parallel-Serien-Umsetzung: Ein Byte wird parallel geladen und Bit für Bit über eine Leitung geschoben. Auf der Gegenseite schiebt ein Register die Bits wieder hinein und gibt sie parallel aus.

Schaltwerke und Speicher Relevanz hoch Quelle: Skript S. 81–84

Halbleiterspeicher (RAM, ROM)

Baut auf: Schaltnetz vs. Schaltwerk, RS-Flip-Flop, Binärpräfixe (kibi, mebi, gibi)

Lernziele

- Ich kann den Matrix-Aufbau eines Speichers erklären und Adressleitungen berechnen.
- Ich kann SRAM und DRAM vergleichen.
- Ich kenne ROM, PROM, EPROM, EEPROM, Flash und NV-RAM.

Die Speicherzellen bilden eine **Matrix**. Ein Teil der Adresse wählt die Zeile, der andere Teil die Spalte. Die Zelle am Kreuzungspunkt wird gelesen oder geschrieben. Das hält die Adressdecodierung klein.

- **16 Ki × 1 Bit:** 2^{14} = 16 384 Zellen, also 128 Zeilen × 128 Spalten. Die Zeile wählen A0–A6, die Spalte A7–A13.
- **2 Ki × 8 Bit** (wortorganisiert): 2048 Wörter zu je 8 Bit, mit 11 Adressleitungen A0–A10.

Typ	Wie wird gespeichert?	Eigenschaften
SRAM	Flip-Flops	schnell, braucht viel Platz
DRAM	Ladung in einem Kondensator (geladen = 1)	muss etwa alle paar ms **aufgefrischt** werden (Refresh), nach dem Lesen neu schreiben, langsamer, aber **hohe Dichte**
Masken-ROM	bei der Herstellung festgelegt	nur bei großen Stückzahlen rentabel
PROM	Verbindungen werden durchgebrannt	einmal programmierbar, nicht umkehrbar
EPROM	Ladung auf einem isolierten Gate	wird mit **UV-Licht** durch ein Fenster gelöscht, einige hundert Mal
EEPROM	elektrisch löschen und schreiben	Umprogrammieren dauert einige ms
Flash	alle Zellen auf einmal elektrisch löschen	am häufigsten, kann in der Schaltung bleiben
NV-RAM	pro Bit eine RAM- und eine EEPROM-Zelle	beim Ausschalten RAM → EEPROM, beim Einschalten zurück

RAM ist **flüchtig**, der Inhalt geht ohne Strom verloren. Alle ROM-Arten sind **nicht flüchtig**.

Ergänzung · Rechenbeispiel Kapazität und Adressleitungen

64 Ki × 16 Bit: 64 Ki = 2^{16} Wörter, also 16 Adressleitungen. Jedes Wort hat 16 Bit, die Gesamtgröße ist $2^{16} \cdot 16 = 2^{20}$ Bit = 2^{17} Byte = 128 KiB.

Faustregel: Adressleitungen = $\log_2$ (Anzahl der Wörter). Datenleitungen = Bits pro Wort. Bei einer Matrix teilt man die Adressleitungen auf Zeile und Spalte auf.

Typische Fehler

- Adressleitungen = Anzahl der Zellen statt $\log_2$.
- EPROM (UV) und EEPROM (elektrisch) verwechselt.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Warum ist DRAM als Arbeitsspeicher üblich? Eine DRAM-Zelle braucht nur einen Transistor und einen Kondensator, eine SRAM-Zelle etwa sechs Transistoren. Dafür ist SRAM schneller – er wird als Cache im Prozessor verwendet. Prüferfrage: „Warum wird beim DRAM nach dem Lesen neu geschrieben?“ – Das Lesen entlädt den Kondensator (zerstörendes Lesen).

Schaltwerke und Speicher Relevanz hoch Quelle: Skript S. 84–88

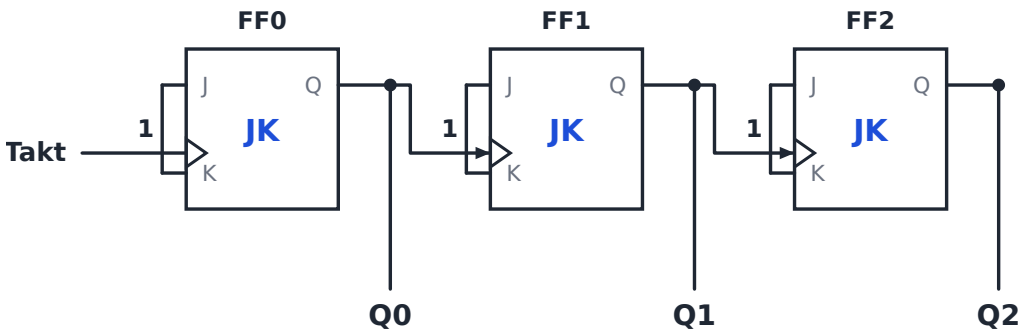
Zähler (synchron, asynchron, Ring)

Baut auf: JK-Flip-Flop, Register

Lernziele

- Ich kann synchrone und asynchrone Dualzähler unterscheiden (Takt, Vor- und Nachteile).
- Ich kann die falschen Zwischenzustände eines Asynchronzählers bestimmen.
- Ich kenne Ringzähler, Rückwärtszähler, ladbare Zähler und Zählererweiterung (EWE/EWA).

- **Ringzähler:** ein rückgekoppeltes Schieberegister, in dem genau eine 1 umläuft (**1-aus-n-Code**). Braucht einen Reset für den Startzustand.
- **Synchroner Dualzähler:**
  - Aus JK-Flip-Flops, die alle **denselben Takt** bekommen.
  - Eine Stelle kippt genau dann, wenn alle vorherigen Stellen 1 sind. Das prüfen UND-Gatter an J und K.
  - Alle Ausgänge ändern sich gleichzeitig, nach genau einer Flip-Flop-Verzögerung.
- **Asynchroner Dualzähler:**
  - Alle Flip-Flops haben J = K = 1. Der Ausgang einer Stelle ist der Takt für die nächste. Eine Stelle kippt, wenn die Stelle davor von 1 auf 0 fällt.
  - Vorteil: einfacher gebaut.
  - Nachteil: falsche Zwischenzustände. Beim Übergang von 7 auf 8 läuft er kurz über 6, 4 und 0, weil sich die Stellen nacheinander ändern.
  - Vor dem Abfragen muss man die maximale Einstellzeit abwarten.
  - Wird für Frequenzteilung und reines Zählen verwendet.
- **Rückwärtszähler:** Eine Stelle kippt, wenn alle vorherigen 0 sind. Dazu verwendet man die $\bar{Q}$-Ausgänge. Mit UP/DOWN̄ kann man umschalten.
- **Ladbarer Zähler:** Bei CT/$\bar{L}$D = 1 zählt er, bei CT/$\bar{L}$D = 0 übernimmt er die Werte an D.
- **BCD-Zähler:** springt nach 9 wieder auf 0.
- **Zählererweiterung:**
  - Asynchron: Der höchste Ausgang ist der Takt für den nächsten Baustein.
  - Synchron: über den Erweiterungseingang EWE und den Erweiterungsausgang EWA. EWA = 1 gilt nur, wenn der Zählerstand 15 ist **und** EWE = 1. Lange Ketten senken die maximale Zählfrequenz.



Der Ausgang einer Stufe ist der Takt der nächsten Stufe

Asynchroner Dualzähler mit 3 JK-Flip-Flops (J = K = 1). Jede Stufe taktet die nächste.

Ergänzung · Zählen und Zwischenzustände

Frequenzteilung: Jede Stufe halbiert die Frequenz. Q0 hat f/2, Q1 hat f/4, Q2 hat f/8.

Zwischenzustände beim Asynchronzähler am Beispiel 3 → 4 (011 → 100): Zuerst fällt Q0 von 1 auf 0 (010). Dieses Fallen taktet Stufe 1, sie kippt von 1 auf 0 (000). Dieses Fallen taktet Stufe 2, sie kippt von 0 auf 1 (100). Kurz sieht man also 010 und 000, bevor 100 steht.

Synchroner Zähler: Eine Stufe kippt, wenn alle Stufen vor ihr 1 sind, also J = K = Q0 & Q1 & … der niedrigeren Stufen. Dann ändern sich alle Ausgänge gleichzeitig.

Typische Fehler

- Beim Asynchronzähler falsche Kipprichtung (er kippt bei 1 → 0 der Vorstufe).
- Ringzähler mit Dualzähler verwechselt.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Ein Asynchronzähler mit n Flip-Flops ist ein Frequenzteiler durch 2^n: Jede Stufe halbiert die Frequenz. Die Zwischenzustände entstehen, weil jede Stufe erst nach der Verzögerung der vorherigen kippt („ripple counter“). Im schlimmsten Fall (111…1 → 000…0) addieren sich alle n Verzögerungen.

Synchrone Schaltwerke entwerfen

Baut auf: D-Latch und D-Flip-Flop, JK-Flip-Flop

Lernziele

- Ich kann den Aufbau (Z, F, G) und das Mealy-Modell erklären.
- Ich kann einen Zustandsgraphen zeichnen (2^k Pfeile pro Zustand).
- Ich kann Zustände codieren und die Übergangstabelle aufstellen.
- Ich kann mit D = Q_neu die Vorbereitungsfunktionen bestimmen.
- Ich kenne Power-On-Reset und Anfangsinitialisierung.

Ein synchrones Schaltwerk besteht aus drei Teilen:

- **Z** = Zustandsspeicher, zum Beispiel aus D-Flip-Flops. Er wechselt seinen Zustand mit dem Takt.
- **G** = Übergangsfunktion: Aus Eingängen und aktuellem Zustand wird der Folgezustand.
- **F** = Ausgabefunktion: Aus Eingängen und aktuellem Zustand werden die Ausgänge.

Das mathematische Modell dafür ist der **Mealy-Automat**.

1. Zustände überlegen und Zustandsgraph zeichnen

Jeder Kreis ist ein Zustand, jeder Pfeil ein Übergang, beschriftet mit `Eingänge / Ausgang`. Bei k Eingängen gehen von **jedem** Zustand genau 2^k Pfeile weg, einer pro Eingangskombination. Schleifen auf denselben Zustand zählen mit.

2. Zustände codieren

Für z Zustände braucht man b Flip-Flops mit $z \leq 2^b$. Welcher Zustand welches Bitmuster bekommt, ist frei wählbar, außer die Angabe schreibt es vor, wie beim Zähler.

3. Übergangstabelle aufstellen

Links stehen alter Zustand und Eingänge, rechts neuer Zustand und Ausgänge. Für alle Kombinationen!

4. Vorbereitung der Flip-Flops bestimmen

Mit D-Flip-Flops: $D = Q_{neu}$. Man nimmt also einfach die Spalte mit dem neuen Zustand.

Mit JK-Flip-Flops gilt diese Tabelle:

$Q_{alt} \rightarrow Q_{neu}$	J	K	J = K (zusammengeschaltet)
0 → 0	0	X	0
0 → 1	1	X	1
1 → 0	X	1	1
1 → 1	X	0	0

Regel: Bei $Q_{alt} = 0$ ist $J = Q_{neu}$ und $K = X$. Bei $Q_{alt} = 1$ ist $J = X$ und $K = \bar{Q}_{neu}$.

5. Mit dem KV-Diagramm vereinfachen und die Schaltung zeichnen

> **Unbenützte Zustände:** Beim Einschalten kann das Schaltwerk zufällig in einem unbenützten Zustand landen. Dagegen gibt es zwei Lösungen:
>
> - **Power-On-Reset:** Eine Zusatzschaltung setzt alle Flip-Flops beim Start über ihren asynchronen Reset in einen definierten Zustand. Dann dürfen die unbenützten Zeilen in der Tabelle **X** sein (frei wählbar, macht die Formeln kürzer).
> - **Anfangsinitialisierung:** Der unbenützte Zustand wird im Graphen eingezeichnet, und alle seine Pfeile führen in den Initialzustand. Dann gibt es keine X-Stellen, und die Formeln werden länger. Der Initialzustand wird erst nach der **ersten Taktflanke** erreicht.

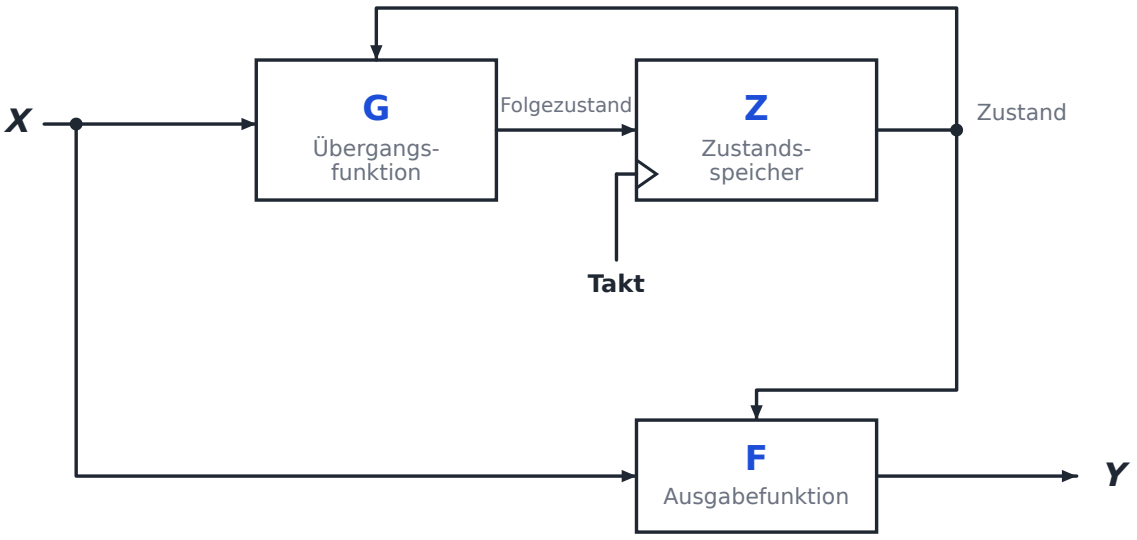
Es gibt drei Zustände: aus, mittel und voll.

- Taster H (hell) → voll
- Taster D (dunkel) → aus
- beide gedrückt → mittel
- keiner gedrückt → der Zustand bleibt
- Bei jeder Änderung piepst P = 1.

Codierung mit Q_0Q_1: aus = 00, mittel = 01, voll = 11. Das Muster 10 bleibt unbenützt und wird mit Power-On-Reset als X behandelt.

alter Zustand	**aus (00)**				**mittel (01)**				**voll (11)**			
D	0	1	0	1	0	1	0	1	0	1	0	1
H	0	0	1	1	0	0	1	1	0	0	1	1
neu	aus	aus	voll	mittel	mittel	aus	voll	mittel	voll	aus	voll	mittel
D_0	0	0	1	0	0	0	1	0	1	0	1	0
D_1	0	0	1	1	1	0	1	1	1	0	1	1
P	0	0	1	1	0	1	1	0	0	1	0	1

$$D_0 = (H \,\&\, \bar{D}) \vee (Q_0 \,\&\, \bar{D})$$
$$D_1 = H \vee (Q_1 \,\&\, \bar{D})$$
$$P = (Q_0 \,\&\, D) \vee (\bar{Q}_1 \,\&\, H) \vee (Q_1 \,\&\, \bar{H} \,\&\, D) \vee (\bar{Q}_0 \,\&\, H \,\&\, \bar{D})$$



Aufbau eines synchronen Schaltwerks: Zustandsspeicher Z, Übergangsfunktion G und Ausgabefunktion F.

Ergänzung · Entwurf als Checkliste

1. Zustände überlegen und den **Zustandsgraphen** zeichnen. Bei k Eingängen gehen von jedem Zustand 2^k Pfeile weg.
2. Zustände **codieren**. Für z Zustände braucht man b Flip-Flops mit $z \leq 2^b$.
3. **Übergangstabelle** für alle Kombinationen aus Zustand und Eingängen aufstellen: neuer Zustand und Ausgänge.
4. Die Flip-Flops **vorbereiten**: Bei D-Flip-Flops ist D der neue Zustand, bei JK gilt die Tabelle von oben.
5. Mit dem **KV-Diagramm** vereinfachen und die Gleichungen für D (bzw. J, K) und die Ausgänge aufstellen.
6. Die **Schaltung** zeichnen.
7. Unbenützte Zustände behandeln (Power-On-Reset oder Anfangsinitialisierung).

Im Bild entspricht G den Gleichungen aus Schritt 5 für die Flip-Flop-Eingänge, F den Gleichungen für die Ausgänge und Z den Flip-Flops selbst.

Typische Fehler

- Nicht für jede Eingangskombination einen Pfeil gezeichnet.
- Unbenützte Zustände ohne Reset als X behandelt.
- Zustand-Codierung beim Zähler frei gewählt, obwohl der Zählerstand ablesbar sein muss.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Mealy vs. Moore: Beim Mealy-Automaten hängt die Ausgabe von Zustand UND Eingang ab, sie steht an den Pfeilen („Eingang / Ausgang“). Beim Moore-Automaten hängt sie nur vom Zustand ab und steht im Kreis. Moore-Ausgänge ändern sich nur mit dem Takt, Mealy-Ausgänge können direkt auf Eingänge reagieren. Das Skript verwendet Mealy (Lichtsteuerung: Piepser hängt vom Übergang ab).

ROM als Logik, maximale Taktfrequenz, CPLD/FPGA

Baut auf: Halbleiterspeicher (RAM, ROM), Synchrone Schaltwerke entwerfen

Lernziele

- Ich kann erklären, wie ein ROM eine logische Funktion realisiert (Eingänge = Adresse).
- Ich kann $f_{max} = 1 / (t_1 + t_2 + t_3)$ berechnen.
- Ich kann CPLD und FPGA unterscheiden.

- **Logik mit ROM:** Die Eingänge werden zur Adresse, das gespeicherte Bit ist der Funktionswert. Man speichert also die ganze Wahrheitstabelle und braucht kein KV-Diagramm. Beispiele: 8 Eingänge → 256 × 1 Bit. Ein Speicher mit 8 Ki × 8 Bit ergibt 8 Funktionen mit je 13 Eingängen (2^{13} = 8 Ki). Mit einem ROM kann man auch die Funktionen F und G eines Schaltwerks bilden.
- **Maximale Taktfrequenz:** Bis zur nächsten Flanke müssen das Flip-Flop schalten (t_1), die Übergangsfunktion rechnen (t_2) und die Setup-Zeit (t_3) eingehalten werden.

  $T_{min} \geq t_1 + t_2 + t_3 \qquad f_{max} = 1 / T_{min}$

- **CPLD:** wenige, aber komplexe Blöcke („viele PALs“). Die Konfiguration liegt im Flash oder EEPROM und ist sofort nach dem Einschalten da.
- **FPGA:** hunderttausende kleine Logikblöcke. Die Konfiguration liegt im **flüchtigen RAM** und muss bei jedem Start neu geladen werden.
- Beide werden mit Hardwarebeschreibungssprachen konfiguriert (zum Beispiel VHDL), auch direkt in der Schaltung (in-system-programming).

Ergänzung · Beispiel: Volladdierer als ROM

Der Volladdierer hat 3 Eingänge (cin, b, a) und 2 Ausgänge (cout, e). Als ROM braucht man also $2^3 = 8$ Adressen mit je 2 Bit. In jeder Zeile steht das Ergebnis aus der Wahrheitstabelle:

Adresse cin b a	**000**	**001**	**010**	**011**	**100**	**101**	**110**	**111**
Inhalt cout e	00	01	01	10	01	10	10	11

Kein KV-Diagramm nötig: Die Wahrheitstabelle wird direkt gespeichert. Der Preis: Die Größe wächst mit 2 hoch Anzahl der Eingänge.

Typische Fehler

- ROM-Größe als Anzahl der Eingänge statt 2^Eingänge.
- In f_max die Setup-Zeit vergessen.

Zum Vertiefen: Hintergründe und Prüfernachfragen

Ein ROM für ein Schaltwerk speichert F und G zusammen: Adresse = Eingänge + aktueller Zustand, Daten = neuer Zustand + Ausgänge. Für die Lichtsteuerung: 4 Adressbits (D, H, Q_0, Q_1) und 3 Datenbits (D_0, D_1, P) → 16 × 3 Bit. Die Komplexität der Funktion ist dann egal – es steht einfach die ganze Wahrheitstabelle im Speicher.